Assignments:

1. Write a program to take a list of 10 farmers, fruits they sell, their cities & the quantity of fruits they have as input. Save the data to notepad before & after sorting based on quantity in tabular manner. Also perform search operation based on farmers' name.

Ans-

```
import java.io.*;

Import java.util.*;


Class Farmer {

  String name;

  String fruit;

  String city;

  Int quantity;


  Farmer(String name, String fruit, String city, int quantity) {

    This.name = name;

    This.fruit = fruit;

    This.city = city;

    This.quantity = quantity;

  }

}


Public class FarmerApp {


  Public static void writeFile(String fileName, List<Farmer> list) throws Exception {

    FileWriter fw = new FileWriter(fileName);

    Fw.write("Name\tFruit\tCity\tQuantity\n");
```

```
    For (Farmer f : list) {
        Fw.write(f.name + “\t” + f.fruit + “\t” + f.city + “\t” + f.quantity + “\n”);
    }
    Fw.close();
}

Public static void main(String[] args) throws Exception {
    Scanner sc = new Scanner(System.in);
    List<Farmer> list = new ArrayList<>();

    // Input 10 farmers
    For (int i = 0; i < 10; i++) {
        System.out.println(“Enter Farmer “ + (i + 1) + “ details:”);

        System.out.print(“Name: “);
        String name = sc.next();

        System.out.print(“Fruit: “);
        String fruit = sc.next();

        System.out.print(“City: “);
        String city = sc.next();

        System.out.print(“Quantity: “);
        Int qty = sc.nextInt();
```

```
    List.add(new Farmer(name, fruit, city, qty));
}

// Save before sorting
writeFile("before_sort.txt", list);

// Sort by quantity
Collections.sort(list, (a, b) → a.quantity – b.quantity);

// Save after sorting
writeFile("after_sort.txt", list);

// Search operation
System.out.print("\nEnter farmer name to search: ");
String search = sc.next();

Boolean found = false;

For (Farmer f : list) {
    If (f.name.equalsIgnoreCase(search)) {
        System.out.println("\n Farmer Found!");
        System.out.println("Name: " + f.name);
        System.out.println("Fruit: " + f.fruit);
        System.out.println("City: " + f.city);
        System.out.println("Quantity: " + f.quantity);
```

```
            Found = true;
            Break;
        }
    }

    If (!found) {
        System.out.println("Farmer not found!");
    }

    Sc.close();
  }
}
```

OUTPUT

Enter Farmer 1 details:

Name: Ram

Fruit: Mango

City: Betul

Quantity: 50

...

File: before_sort.txt

Name	Fruit	City	Quantity
Ram	Mango	Betul	50
Shyam	Apple	Indore	20
Mohan	Banana	Bhopal	80

File: after_sort.txt (Sorted by Quantity)

Name Fruit City Quantity

Shyam Apple Indore 20

Ram Mango Betul 50

Mohan Banana Bhopal 80

Search Output Example

Enter farmer name to search: Ram

Farmer Found!

Name: Ram

Fruit: Mango

City: Betul

Quantity: 50

2. Write a program to take quantity & quality of fruits Mango, Apple & Papaya as input. Quality may be good, bad or average. If entered quantity of fruit is greater than 1000, store it as half. Perform search for a given fruit with given quality.

Ans-

```
Import java.util.*;


Class Fruit {

  String name;

  Int quantity;

  String quality;
```

```
  Fruit(String name, int quantity, String quality) {
    This.name = name;
    This.quantity = quantity;
    This.quality = quality;
  }
}

Public class FruitApp {

  Public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);

    Fruit[] fruits = new Fruit[3];

    // Input for 3 fruits
    For (int i = 0; i < 3; i++) {
      System.out.println("Enter details for fruit " + (i + 1));

      System.out.print("Fruit name (Mango/Apple/Papaya): ");
      String name = sc.next();

      System.out.print("Quantity: ");
      Int qty = sc.nextInt();
```

```
    System.out.print("Quality (good/bad/average): ");
    String quality = sc.next();

    // Condition: if quantity > 1000, store half
    If (qty > 1000) {
        Qty = qty / 2;
    }

    Fruits[i] = new Fruit(name, qty, quality);
}

// Search operation
System.out.print("\nEnter fruit name to search: ");
String searchName = sc.next();

System.out.print("Enter quality to search: ");
String searchQuality = sc.next();

Boolean found = false;

For (Fruit f : fruits) {
    If (f.name.equalsIgnoreCase(searchName)
```

```
            && f.quality.equalsIgnoreCase(searchQuality)) {

          System.out.println("\n Fruit Found!");
          System.out.println("Name: " + f.name);
          System.out.println("Quantity: " + f.quantity);
          System.out.println("Quality: " + f.quality);

          Found = true;
          Break;
        }
      }

      If (!found) {
        System.out.println("Fruit not found!");
      }

      Sc.close();
    }
}
```

OUTPUT

Fruit Found!

Name: Mango

Quantity: 600

Quality: good

3. There are four orchards, namely A, B, C & D with fruits in them as Apple, Litchi, Mango, Apple respectively. The qualities of the fruits in the orchards are good, good, good & bad respectively.

The available orchards in a tree-view.

Search for a given fruit in the available orchards after taking input from the user & print the tree.

Ans-

```
Import java.util.*;



Class Orchard {

  String name;

  String fruit;

  String quality;

  List<Orchard> children;



  Orchard(String name, String fruit, String quality) {

    This.name = name;

    This.fruit = fruit;

    This.quality = quality;

    This.children = new ArrayList<>();

  }



  Void addChild(Orchard child) {
```

```
    Children.add(child);
  }
}

Public class OrchardTree {

  // Print tree structure
  Public static void printTree(Orchard node, String space) {
    If (node == null) return;

    System.out.println(space + "Orchard " + node.name +
        " → Fruit: " + node.fruit +
        " | Quality: " + node.quality);

    For (Orchard child : node.children) {
      printTree(child, space + "   ");
    }
  }

  // Search fruit in tree
  Public static boolean search(Orchard node, String fruit) {
    If (node == null) return false;
```

```
    If (node.fruit.equalsIgnoreCase(fruit)) {
        System.out.println("\n Fruit Found in Orchard " + node.name);
        System.out.println("Fruit: " + node.fruit);
        System.out.println("Quality: " + node.quality);
        Return true;
    }

    For (Orchard child : node.children) {
        If (search(child, fruit)) return true;
    }

    Return false;
}

Public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);

    // Root (can be a virtual node)
    Orchard root = new Orchard("ROOT", "-", "-");

    // Creating orchards
    Orchard A = new Orchard("A", "Apple", "Good");
    Orchard B = new Orchard("B", "Litchi", "Good");
```

```
Orchard C = new Orchard("C", "Mango", "Good");

Orchard D = new Orchard("D", "Apple", "Bad");


// Building tree

Root.addChild(A);

Root.addChild(B);

Root.addChild(C);

Root.addChild(D);


// Print tree

System.out.println("🌳 Orchard Tree Structure:\n");

printTree(root, "");


// Search

System.out.print("\nEnter fruit to search: ");

String fruit = sc.next();


Boolean found = search(root, fruit);


If (!found) {

   System.out.println("\n Fruit not found in any orchard!");

}
```

```
    Sc.close();
  }
}
```

OUTPUT

Fruit Found in Orchard A

Fruit: Apple

Quality: Good

4.Take a map provided as a graph & display all the subtrees of connected points.

```
Import java.util.*;


Public class GraphSubtrees {


  Static void dfs(int node, boolean[] visited, List<List<Integer≫ adj, List<Integer>
component) {
    Visited[node] = true;
    Component.add(node);


    For (int neighbor : adj.get(node)) {
      If (!visited[neighbor]) {
        Dfs(neighbor, visited, adj, component);
      }
    }
  }
```

```
Public static void main(String[] args) {
  Scanner sc = new Scanner(System.in);

  System.out.print("Enter number of nodes: ");
  Int n = sc.nextInt();

  System.out.print("Enter number of edges: ");
  Int e = sc.nextInt();

  List<List<Integer≫ adj = new ArrayList<>();

  For (int i = 0; i ≤ n; i++) {
    Adj.add(new ArrayList<>());
  }

  System.out.println("Enter edges (u v):");
  For (int i = 0; i < e; i++) {
    Int u = sc.nextInt();
    Int v = sc.nextInt();

    Adj.get(u).add(v);
    Adj.get(v).add(u); // undirected graph
```

```
        }

        Boolean[] visited = new boolean[n + 1];

        System.out.println("\n🌐 Subtrees (Connected Components):\n");

        Int count = 1;

        For (int i = 1; i ≤ n; i++) {
          If (!visited[i]) {
            List<Integer> component = new ArrayList<>();

            Dfs(i, visited, adj, component);

            System.out.println("Subtree " + count + ": " + component);
            Count++;
          }
        }

        Sc.close();
      }
    }
```

Output-

Enter number of nodes: 5

Enter number of edges: 2

Enter edges (u v):

1 2

3 4

🌐 Subtrees (Connected Components):

Subtree 1: [1, 2]

Subtree 2: [3, 4]

Subtree 3: [5]

5. Enter entry point & goal point & produce the matching subtree of the graph.

```
Import java.util.*;


Public class GraphEntryGoal {


   Static boolean found = false;


   Static void dfs(int node, int goal, boolean[] visited,

               List<List<Integer>> adj, List<Integer> path) {


      If (found) return;
```

```
    Visited[node] = true;
    Path.add(node);

    // If goal found
    If (node == goal) {
        Found = true;
        Return;
    }

    For (int neighbor : adj.get(node)) {
        If (!visited[neighbor]) {
            Dfs(neighbor, goal, visited, adj, path);

            If (found) return; // stop further traversal
        }
    }

    // backtrack if not part of solution path
    If (!found) {
        Path.remove(path.size() – 1);
    }
}
```

```
Public static void main(String[] args) {
  Scanner sc = new Scanner(System.in);

  System.out.print("Enter number of nodes: ");
  Int n = sc.nextInt();

  System.out.print("Enter number of edges: ");
  Int e = sc.nextInt();

  List<List<Integer>> adj = new ArrayList<>();

  For (int i = 0; i ≤ n; i++) {
    Adj.add(new ArrayList<>());
  }

  System.out.println("Enter edges (u v):");
  For (int i = 0; i < e; i++) {
    Int u = sc.nextInt();
    Int v = sc.nextInt();

    Adj.get(u).add(v);
    Adj.get(v).add(u); // undirected graph
  }
```

```
        System.out.print("Enter entry point: ");
        Int start = sc.nextInt();

        System.out.print("Enter goal point: ");
        Int goal = sc.nextInt();

        Boolean[] visited = new boolean[n + 1];
        List<Integer> path = new ArrayList<>();

        Dfs(start, goal, visited, adj, path);

        System.out.println("\n🌳 Matching Subtree / Path:");

        If (found) {
            System.out.println(path);
        } else {
            System.out.println("Goal not reachable from entry point!");
        }

        Sc.close();
    }
}
```

OUTPUT

🌳 **Matching Subtree / Path:**

[1, 2, 3, 4, 5]

6. Give the weightage for each edge of nodes, and produce the shortest path from start to end node.

Ans-import java.util.*;

```
Class Edge {

  Int to, weight;


  Edge(int to, int weight) {

    This.to = to;

    This.weight = weight;

  }

}


Public class ShortestPath {


  Static class Node implements Comparable<Node> {

    Int node, dist;


    Node(int node, int dist) {
```

```
        This.node = node;
        This.dist = dist;
    }

    Public int compareTo(Node other) {
        Return this.dist – other.dist;
    }
}

Public static void dijkstra(int start, int end, List<List<Edge≫ graph, int n) {

    Int[] dist = new int[n + 1];
    Int[] parent = new int[n + 1];
    Boolean[] visited = new boolean[n + 1];

    Arrays.fill(dist, Integer.MAX_VALUE);

    PriorityQueue<Node> pq = new PriorityQueue<>();

    Dist[start] = 0;
    Pq.add(new Node(start, 0));
    Parent[start] = -1;
```

```
While (!pq.isEmpty()) {
   Node current = pq.poll();
   Int u = current.node;

   If (visited[u]) continue;
   Visited[u] = true;

   For (Edge edge : graph.get(u)) {
      Int v = edge.to;
      Int w = edge.weight;

      If (dist[u] + w < dist[v]) {
         Dist[v] = dist[u] + w;
         Pq.add(new Node(v, dist[v]));
         Parent[v] = u;
      }
   }
}

// Print shortest distance
System.out.println("\nShortest distance from " + start + " to " + end + " = " + dist[end]);

// Reconstruct path
```

```
    List<Integer> path = new ArrayList<>();

    For (int v = end; v != -1; v = parent[v]) {
        Path.add(v);
    }

    Collections.reverse(path);

    System.out.println("Shortest Path: " + path);
}

Public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);

    System.out.print("Enter number of nodes: ");
    Int n = sc.nextInt();

    System.out.print("Enter number of edges: ");
    Int e = sc.nextInt();

    List<List<Edge≫ graph = new ArrayList<>();

    For (int i = 0; i ≤ n; i++) {
```

```
    Graph.add(new ArrayList<>());
}

System.out.println("Enter edges (u v weight):");

For (int i = 0; i < e; i++) {
    Int u = sc.nextInt();
    Int v = sc.nextInt();
    Int w = sc.nextInt();

    Graph.get(u).add(new Edge(v, w));
    Graph.get(v).add(new Edge(u, w)); // undirected graph
}

System.out.print("Enter start node: ");
Int start = sc.nextInt();

System.out.print("Enter end node: ");
Int end = sc.nextInt();

Dijkstra(start, end, graph, n);

Sc.close();
```

```
    }
}
```

Output-

Shortest distance from 1 to 5 = 6

Shortest Path: [1, 2, 3, 5]

7.Write a program to demonstrate the use of all keywords made for exception handling.

Ans-import java.util.*;

```
public class ExceptionDemo {

    // throws keyword used here
    static void checkAge(int age) throws Exception {

        // throw keyword used here
        if (age < 18) {
            throw new Exception("Age is less than 18, not allowed!");
        } else {
            System.out.println("Valid age: " + age);
        }
    }

    public static void main(String[] args) {
```

```
        Scanner sc = new Scanner(System.in);

        try {
            System.out.print("Enter age: ");
            int age = sc.nextInt();

            checkAge(age); // may throw exception

        } catch (Exception e) {
            // catch block
            System.out.println("Exception caught: " + e.getMessage());

        } finally {
            // finally block always executes
            System.out.println("Program ended (finally block executed)");
            sc.close();
        }
    }
}
```

Output-Enter age: 15

Exception caught: Age is less than 18, not allowed!

Program ended (finally block executed)

8. Write a program to withraw money from a bank account, if the withdrawn money is greater than the balance, then it throws NoMoneyException.

Ans-import java.util.*;

```
// Custom Exception class

Class NoMoneyException extends Exception {

   NoMoneyException(String message) {

      Super(message);

   }

}


Public class BankAccount {


   Static int balance = 5000;


   Static void withdraw(int amount) throws NoMoneyException {


      If (amount > balance) {

         Throw new NoMoneyException("Insufficient Balance! Withdrawal not allowed.");

      } else {

         Balance = balance – amount;

         System.out.println("Withdrawal successful!");

         System.out.println("Remaining Balance: " + balance);
```

```
    }
  }

  Public static void main(String[] args) {

    Scanner sc = new Scanner(System.in);

    Try {
      System.out.println("Current Balance: " + balance);
      System.out.print("Enter amount to withdraw: ");
      Int amount = sc.nextInt();

      Withdraw(amount);

    } catch (NoMoneyException e) {
      System.out.println("Exception: " + e.getMessage());

    } finally {
      System.out.println("Transaction Completed (finally block executed)");
      Sc.close();
    }
  }
}
```

OUTPUT-

Case 1: Valid Withdrawal

Plain text

Current Balance: 5000

Enter amount to withdraw: 2000

Withdrawal successful!

Remaining Balance: 3000

Transaction Completed (finally block executed)

Case 2: Invalid Withdrawal

Plain text

Current Balance: 5000

Enter amount to withdraw: 7000

Exception: Insufficient Balance! Withdrawal not allowed.

Transaction Completed (finally block executed)

9. Write a program which enables to use a platform by a train one at a time.

```
Class Platform {


  // synchronized method ensures only one train at a time

  Synchronized void usePlatform(String trainName) {


    System.out.println(trainName + " is entering the platform");


    Try {
```

```
        Thread.sleep(2000); // train is on platform
      } catch (InterruptedException e) {
        System.out.println(e);
      }

      System.out.println(trainName + " is leaving the platform\n");
    }
}

Class Train extends Thread {
    Platform p;
    String name;

    Train(Platform p, String name) {
      This.p = p;
      This.name = name;
    }

    Public void run() {
      p.usePlatform(name);
    }
}
```

```
Public class TrainStation {

   Public static void main(String[] args) {

      Platform platform = new Platform();

      Train t1 = new Train(platform, “Train A”);
      Train t2 = new Train(platform, “Train B”);
      Train t3 = new Train(platform, “Train C”);

      T1.start();
      T2.start();
      T3.start();
   }
}
```

Output-Train A is entering the platform

Train A is leaving the platform

Train C is entering the platform

Train C is leaving the platform

Train B is entering the platform

Train B is leaving the platform

10. Write a programme of Restaurant and Waiter, which accept order if kitchen if free and sheff deliver orders if he is free.

Ans-class Kitchen {

```
  boolean isBusy = false;

  // synchronized ensures only one order at a time
  synchronized void takeOrder(String waiterName, String order) {

    if (isBusy) {
      System.out.println("Kitchen is busy. " + waiterName + " waiting...");
      try {
        wait();
      } catch (InterruptedException e) {
        System.out.println(e);
      }
    }

    isBusy = true;

    System.out.println(waiterName + " took order: " + order);

    try {
```

```
            Thread.sleep(2000); // time to prepare order
        } catch (InterruptedException e) {
            System.out.println(e);
        }

        System.out.println("Order prepared for: " + order);

        isBusy = false;
        notify(); // notify waiter/chef that kitchen is free
    }
}

class Waiter extends Thread {

    Kitchen kitchen;
    String order;

    Waiter(Kitchen kitchen, String order) {
        this.kitchen = kitchen;
        this.order = order;
    }

    public void run() {
```

```
        kitchen.takeOrder(Thread.currentThread().getName(), order);
    }
}

public class Restaurant {

    public static void main(String[] args) {

        Kitchen kitchen = new Kitchen();

        Waiter w1 = new Waiter(kitchen, "Pizza");
        Waiter w2 = new Waiter(kitchen, "Burger");
        Waiter w3 = new Waiter(kitchen, "Pasta");

        w1.setName("Waiter 1");
        w2.setName("Waiter 2");
        w3.setName("Waiter 3");

        w1.start();
        w2.start();
        w3.start();
    }
}
```

OUTPUT-

Waiter 1 took order: Pizza

Order prepared for: Pizza

Waiter 2 took order: Burger

Order prepared for: Burger

Waiter 3 took order: Pasta

Order prepared for: Pasta